

Subset Sum; Shortest Paths

1 Subset Sum and Knapsack

- a scheduling problem
- only natural numbers in the input data
- the subset sum algorithm
- the knapsack problem

2 Shortest Paths in a Graph

- directed graphs with negative weights
- the Bellman-Ford algorithm

CS 401/MCS 401 Lecture 14
Computer Algorithms I
Jan Verschelde, 20 July 2026

Subset Sum; Shortest Paths

1 Subset Sum and Knapsack

- a scheduling problem
- only natural numbers in the input data
- the subset sum algorithm
- the knapsack problem

2 Shortest Paths in a Graph

- directed graphs with negative weights
- the Bellman-Ford algorithm

a scheduling problem

Consider a resource, available between time 0 and $\mathbb{W}$.

We have n requests $\{ 1, 2, \dots, n \}$ for the resource.

Denote the set of requests by $R = \{ 1, 2, \dots, n \}$.

Request i has weight w_i , the time it requests on the resource.

Determine the subset of R to compute the maximum sum of weights

$$\begin{aligned} & \max_{S \subseteq R} \sum_{i \in S} w_i \\ & \text{subject to } \sum_{i \in S} w_i \leq \mathbb{W}. \end{aligned}$$

This constrained optimization problem is known as
the subset sum problem.

let us try a greedy rule

Requests $R = \{ 1, 2, \dots, n \}$, with weights w_i and bound $\mathbb{W}$.
Determine the subset of R to compute the maximum sum of weights

$$\max_{S \subseteq R} \sum_{i \in S} w_i \quad \text{subject to} \quad \sum_{i \in S} w_i \leq \mathbb{W}.$$

Sort the weights of the requests, in decreasing order.

Greedy rule: *select heaviest request first.*

Consider $R = \{ \mathbb{W}/2 + 1, \mathbb{W}/2, \mathbb{W}/2 \}$, and apply the rule:

- 1 Select $\mathbb{W}/2 + 1$ first.
- 2 Not possible to select $\mathbb{W}/2$ because $\mathbb{W}/2 + 1 + \mathbb{W}/2 > \mathbb{W}$.
- 3 Selecting $\mathbb{W}/2$ and $\mathbb{W}/2$ is the optimal choice.

This rule fails!

let us try another greedy rule

Requests $R = \{ 1, 2, \dots, n \}$, with weights w_i and bound $\mathbb{W}$.
Determine the subset of R to compute the maximum sum of weights

$$\max_{S \subseteq R} \sum_{i \in S} w_i \quad \text{subject to} \quad \sum_{i \in S} w_i \leq \mathbb{W}.$$

Sort the weights of the requests, in increasing order.

Greedy rule: *select lightest request first.*

Consider $R = \{ 1, \mathbb{W}/2, \mathbb{W}/2 \}$, and apply the rule:

- 1 Select 1 first and then select $\mathbb{W}/2$.
- 2 Not possible to select $\mathbb{W}/2$ because $1 + \mathbb{W}/2 + \mathbb{W}/2 > \mathbb{W}$.
- 3 Selecting $\mathbb{W}/2$ and $\mathbb{W}/2$ is the optimal choice.

This rule fails!

guidelines for dynamic programming

We need dynamic programming when greedy rules do not work.

For dynamic programming to apply, we need a collection of subproblems derived from the original problem that satisfies the properties:

- 1 There are a polynomial number of subproblems.
- 2 The solution to the original problem can be easily computed from the solutions to the subproblems.
- 3 There is a natural order from smallest to largest subproblem, jointly with an easy-to-compute recurrence.

While dynamic programming resembles divide and conquer, if not carefully implemented, it may not lead to an efficient algorithm.

Does weighted interval scheduling apply?

Recall the weighted interval scheduling:

Input: n requests (s_i, f_i, v_i) , $i = 1, 2, \dots, n$,

s_i is the start time of request i ,

f_i is the finish time of request i ,

v_i is the value of request i .

The requests are sorted: $f_1 \leq f_2 \leq \dots \leq f_n$.

For this problem, once a request $[s_i, f_i]$ is selected, automatically all conflicting intervals $[s_j, f_j]$ are excluded in the remaining subproblems left to consider.

For the subset sum problem, once we select a weight, we are left with all remaining $n - 1$ requests.

We have to find a better way to explore the problem space.

Subset Sum; Shortest Paths

1 Subset Sum and Knapsack

- a scheduling problem
- only natural numbers in the input data
- the subset sum algorithm
- the knapsack problem

2 Shortest Paths in a Graph

- directed graphs with negative weights
- the Bellman-Ford algorithm

only natural numbers in the input data

If all weights are positive natural numbers,
then we can work with a threshold $w \leq \mathbb{W}$ to define subproblems.

Given w , $0 \leq w \leq \mathbb{W}$, and
determine the subset of R_k , $R_k = \{ 1, 2, \dots, k \}$,
with maximum sum of weights over the first k requests:

$$\begin{aligned} \text{OPT}(k, w) = \max_{S \subseteq R_k} \sum_{i \in S} w_i \\ \text{subject to } \sum_{i \in S} w_i \leq w. \end{aligned}$$

Our goal is to determine S for $\text{OPT}(n, \mathbb{W})$.

exploring a dichotomy

Denote an optimal solution by $\mathcal{O} \subseteq \{1, 2, \dots, n\}$.

Consider n , the last request. We have two cases:

- 1 Either $n \notin \mathcal{O}$, and then

$$\text{OPT}(n, \mathbb{W}) = \text{OPT}(n-1, \mathbb{W}).$$

We then simply ignore the n -th request.

- 2 Or $n \in \mathcal{O}$, and then

$$\text{OPT}(n, \mathbb{W}) = w_n + \text{OPT}(n-1, \mathbb{W} - w_n).$$

The above formula is wrong when $\mathbb{W} - w_n < 0$.

In case $\mathbb{W} - w_n < 0$, then $n \notin \mathcal{O}$.

So the first case applies always when $w_n > \mathbb{W}$.

a recurrence

For w , $0 \leq w \leq \mathbb{W}$, and $R_k = \{1, 2, \dots, k\}$, denote

$$\begin{aligned} \text{OPT}(k, w) = & \max_{S \subseteq R_k} \sum_{i \in S} w_i \\ & \text{subject to } \sum_{i \in S} w_i \leq w. \end{aligned}$$

The dichotomy leads to the recurrence formulated in a lemma:

Lemma (1)

If $w_k > w$, then

$$\text{OPT}(k, w) = \text{OPT}(k - 1, w),$$

else

$$\text{OPT}(k, w) = \max(\text{OPT}(k - 1, w), w_k + \text{OPT}(k - 1, w - w_k)).$$

Subset Sum; Shortest Paths

1 Subset Sum and Knapsack

- a scheduling problem
- only natural numbers in the input data
- **the subset sum algorithm**
- the knapsack problem

2 Shortest Paths in a Graph

- directed graphs with negative weights
- the Bellman-Ford algorithm

the subset sum algorithm

Subset-Sum($n, \mathbb{W}, w_i$)

Input: n , the number of requests,

$\mathbb{W}$, the bound on the sum of weights,

w_i , weights of all requests $i = 1, 2, \dots, n$.

Output: returns $\text{OPT}(n, \mathbb{W})$.

let $M[0 \dots n, 0 \dots \mathbb{W}]$ be an array of arrays

for $k = 0, 1, \dots, n$ do $M[k, 0] := 0$

for $w = 0, 1, \dots, \mathbb{W}$ do $M[0, w] := 0$

for $k = 1, 2, \dots, n$ do

 for $w = 0, 1, \dots, \mathbb{W}$ do

 if $w_k > w$ then

$M[k, w] := M[k - 1, w]$

 else

$M[k, w] := \max(M[k - 1, w], w_k + M[k - 1, w - w_k])$

return $M[n, \mathbb{W}]$.

a numerical example

Input: $w_1 = 2$, $w_2 = 2$, $w_3 = 3$, $\mathbb{W} = 6$.

Step 0:

item	weights						
	0	1	2	3	4	5	6
3	0						
2	0						
1	0						
0	0	0	0	0	0	0	0

a numerical example

Input: $w_1 = 2$, $w_2 = 2$, $w_3 = 3$, $\mathbb{W} = 6$.

Step 1:

item	weights						
	0	1	2	3	4	5	6
3	0						
2	0						
1	0	0	2	2	2	2	2
0	0	0	0	0	0	0	0

a numerical example

Input: $w_1 = 2$, $w_2 = 2$, $w_3 = 3$, $\mathbb{W} = 6$.

Step 2:

item	weights						
	0	1	2	3	4	5	6
3	0						
2	0	0	2	2	4	4	4
1	0	0	2	2	2	2	2
0	0	0	0	0	0	0	0

a numerical example

Input: $w_1 = 2$, $w_2 = 2$, $w_3 = 3$, $\mathbb{W} = 6$.

Step 3:

item	weights						
	0	1	2	3	4	5	6
3	0	0	2	2	4	5	5
2	0	0	2	2	4	4	4
1	0	0	2	2	2	2	2
0	0	0	0	0	0	0	0

recovering the optimal solution

Exercise 1: on the example with

input: $w_1 = 2$, $w_2 = 2$, $w_3 = 3$, $\mathbb{W} = 6$,

demonstrate the recovery of the optimal solution,
given the output table M .

Formulate the algorithm to recover the optimal solution, given M .

What is the running time of this algorithm?

analysis of the subset sum algorithm

Theorem (2)

For n requests with bound $\mathbb{W}$, $\mathbb{W}$ is a natural number, `Subset-Sum` returns $\text{OPT}(n, \mathbb{W})$ in $O(n\mathbb{W})$ time.

The correctness is proven by induction on n .
The cost follows from the size of M .

Corollary (3)

Given the table M computed by `Subset-Sum`, the optimal set can be computed in $O(n)$ time.

Even as the size of M is $n\mathbb{W}$,
the computation of the optimal set follows one path in the table
and the size of the optimal set is bounded by n .

on the space requirement for subset sum

Exercise 2: Consider six requests:

$w_1 = 33$, $w_2 = 11$, $w_3 = 27$, $w_4 = 32$, $w_5 = 37$, $w_6 = 31$, and $\mathbb{W} = 100$.

- 1 Running the subset sum algorithm, what do you observe on the number of nonzero elements in M , versus the entire size of M ?
- 2 Describe an efficient data structure to efficiently store only the nonzero elements needed in the algorithm.

Subset Sum; Shortest Paths

1 Subset Sum and Knapsack

- a scheduling problem
- only natural numbers in the input data
- the subset sum algorithm
- the knapsack problem

2 Shortest Paths in a Graph

- directed graphs with negative weights
- the Bellman-Ford algorithm

an extension: the knapsack problem

Consider a resource, available between time 0 and $\mathbb{W}$.

We have n requests $\{ 1, 2, \dots, n \}$ for the resource.

Denote the set of requests by $R = \{ 1, 2, \dots, n \}$.

Each request i has

- a weight w_i , the time it requests on the resource, and
- a value v_i , relative to the importance of the request.

Determine the subset of R to compute the maximum sum of values

$$\begin{aligned} \max_{S \subseteq R} \sum_{i \in S} v_i \\ \text{subject to } \sum_{i \in S} w_i \leq \mathbb{W}. \end{aligned}$$

This constrained optimization problem is known as
the knapsack problem.

adjusting the dichotomy for knapsack

Denote an optimal solution by $\mathcal{O} \subseteq \{1, 2, \dots, n\}$.

Consider n , the last request. We have two cases:

- 1 Either $n \notin \mathcal{O}$, and then

$$\text{OPT}(n, \mathbb{W}) = \text{OPT}(n-1, \mathbb{W}).$$

We then simply ignore the n -th request.

- 2 Or $n \in \mathcal{O}$, and then

$$\text{OPT}(n, \mathbb{W}) = v_n + \text{OPT}(n-1, \mathbb{W} - w_n).$$

The above formula is wrong when $\mathbb{W} - w_n < 0$.

In case $\mathbb{W} - w_n < 0$, then $n \notin \mathcal{O}$.

So the first case applies always when $w_n > \mathbb{W}$.

a recurrence for the knapsack problem

For w , $0 \leq w \leq \mathbb{W}$, and $R_k = \{ 1, 2, \dots, k \}$, denote

$$\begin{aligned} \text{OPT}(k, w) = & \max_{S \subseteq R_k} \sum_{i \in S} v_i \\ & \text{subject to } \sum_{i \in S} w_i \leq w. \end{aligned}$$

The dichotomy leads to the recurrence formulated in a lemma:

Lemma (4)

If $w_k > w$, then

$$\text{OPT}(k, w) = \text{OPT}(k - 1, w),$$

else

$$\text{OPT}(k, w) = \max(\text{OPT}(k - 1, w), v_k + \text{OPT}(k - 1, w - w_k)).$$

the knapsack algorithm

Knapsack($n, \mathbb{W}, w_i, v_i$)

Input: n , the number of requests,
 $\mathbb{W}$, the bound on the sum of weights,
 w_i , weights of all requests $i = 1, 2, \dots, n$,
 v_i , values of all requests $i = 1, 2, \dots, n$.

Output: returns $\text{OPT}(n, \mathbb{W})$.

let $M[0 \dots n, 0 \dots \mathbb{W}]$ be an array of arrays

for $k = 0, 1, \dots, n$ do $M[k, 0] := 0$

for $w = 0, 1, \dots, \mathbb{W}$ do $M[0, w] := 0$

for $k = 1, 2, \dots, n$ do

 for $w = 0, 1, \dots, \mathbb{W}$ do

 if $w_k > w$ then

$M[k, w] := M[k - 1, w]$

 else

$M[k, w] := \max(M[k - 1, w], v_k + M[k - 1, w - w_k])$

return $M[n, \mathbb{W}]$.

a numerical example

Input: $w_1 = 2$, $w_2 = 1$, $w_3 = 3$, $w_4 = 2$, $W = 5$,
 $v_1 = 12$, $v_2 = 10$, $v_3 = 20$, $v_4 = 15$.

Step 0:

item	weights					
	0	1	2	3	4	5
4	0					
3	0					
2	0					
1	0					
0	0	0	0	0	0	0

a numerical example

Input: $w_1 = 2$, $w_2 = 1$, $w_3 = 3$, $w_4 = 2$, $W = 5$,
 $v_1 = 12$, $v_2 = 10$, $v_3 = 20$, $v_4 = 15$.

Step 1:

item	weights					
	0	1	2	3	4	5
4	0					
3	0					
2	0					
1	0	0	12	12	12	12
0	0	0	0	0	0	0

a numerical example

Input: $w_1 = 2$, $w_2 = 1$, $w_3 = 3$, $w_4 = 2$, $W = 5$,
 $v_1 = 12$, $v_2 = 10$, $v_3 = 20$, $v_4 = 15$.

Step 2:

item	weights					
	0	1	2	3	4	5
4	0					
3	0					
2	0	10	12	22	22	22
1	0	0	12	12	12	12
0	0	0	0	0	0	0

a numerical example

Input: $w_1 = 2$, $w_2 = 1$, $w_3 = 3$, $w_4 = 2$, $W = 5$,
 $v_1 = 12$, $v_2 = 10$, $v_3 = 20$, $v_4 = 15$.

Step 3:

item	weights					
	0	1	2	3	4	5
4	0					
3	0	10	12	22	30	32
2	0	10	12	22	22	22
1	0	0	12	12	12	12
0	0	0	0	0	0	0

a numerical example

Input: $w_1 = 2$, $w_2 = 1$, $w_3 = 3$, $w_4 = 2$, $W = 5$,
 $v_1 = 12$, $v_2 = 10$, $v_3 = 20$, $v_4 = 15$.

Step 4:

item	weights					
	0	1	2	3	4	5
4	0	10	15	25	30	37
3	0	10	12	22	30	32
2	0	10	12	22	22	22
1	0	0	12	12	12	12
0	0	0	0	0	0	0

recovering the optimal solution

Exercise 3: on the example with

input: $w_1 = 2, w_2 = 1, w_3 = 3, w_4 = 2, W = 5,$
 $v_1 = 12, v_2 = 10, v_3 = 20, v_4 = 15,$

demonstrate the recovery of the optimal solution,
given the output table M .

Formulate the algorithm to recover the optimal solution, given M .

What is the running time of this algorithm?

analysis of the knapsack algorithm

Theorem (5)

For n requests with bound $\mathbb{W}$, $\mathbb{W}$ is a natural number, Knapsack returns $\text{OPT}(n, \mathbb{W})$ in $O(n\mathbb{W})$ time.

The correctness is proven by induction on n .
The cost follows from the size of M .

Corollary (6)

Given the table M computed by Knapsack, the optimal set can be computed in $O(n)$ time.

Even as the size of M is $n\mathbb{W}$,
the computation of the optimal set follows one path in the table
and the size of the optimal set is bounded by n .

Subset Sum; Shortest Paths

1 Subset Sum and Knapsack

- a scheduling problem
- only natural numbers in the input data
- the subset sum algorithm
- the knapsack problem

2 Shortest Paths in a Graph

- directed graphs with negative weights
- the Bellman-Ford algorithm

CS 401/MCS 401 Lecture 14
Computer Algorithms I
Jan Verschelde, 20 July 2026

Subset Sum; Shortest Paths

1 Subset Sum and Knapsack

- a scheduling problem
- only natural numbers in the input data
- the subset sum algorithm
- the knapsack problem

2 Shortest Paths in a Graph

- directed graphs with negative weights
- the Bellman-Ford algorithm

shortest paths in a graph

Given is a weighted graph $G = (V, E)$
where each edge $(i, j) \in E$ has a cost $c_{i,j}$,
the cost of going directly from vertex i to j .

Costs can be negative. *A negative cost is a profit.*

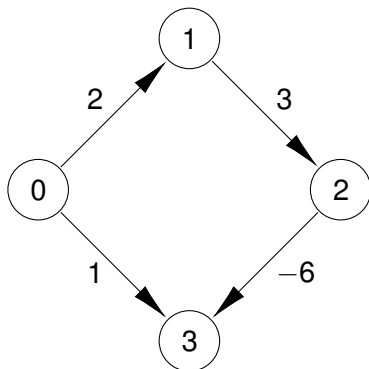
The problem is to compute shortest paths in G .

Does Dijkstra's algorithm work on graphs with negative costs?

Let us compute an example ...

Why would Dijkstra's algorithm not work?

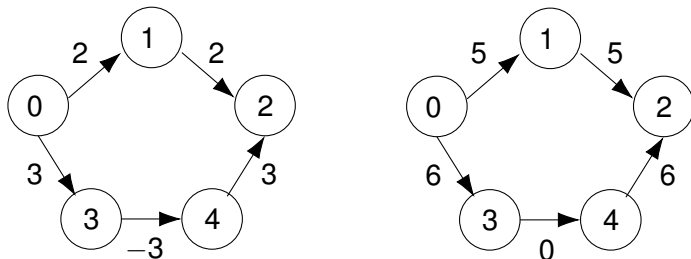
Compute all shortest paths from 0 to all other nodes in the graph:



The shortest path from 0 to 3 goes via 1 and 2.

Can we make the weights positive?

If the smallest weight is $-w$, for $w > 0$,
then add w to all weights in the graph, as illustrated below:



The shortest path from 0 to 2

- 1 has length 3 in the graph at the left and passed through 3 and 4,
- 2 has length 10 in the graph at the right and passed through 1.

The greedy algorithm of Dijkstra does not apply.

find a negative cycle

Definition (7)

Given a graph $G = (V, E)$ with weights $c_{i,j}$ on edges $(i, j) \in E$,
a negative cycle C is a sequence of edges such that

1 the edges in C define a cycle, and

2

$$\sum_{(i,j) \in C} c_{i,j} < 0.$$

Consider a sequence of financial transactions:

- 1 buy from i_1 ,
- 2 sell to i_2 , sell to i_3 , etc,
- 3 return to i_1 with a net profit.

A negative cycle is viewed as a good arbitrage opportunity.

an exercise

Exercise 4:

Consider a directed graph $G = (V, E)$ with weights $c_{i,j}$ on edges $(i, j) \in E$.

- 1 Give an example of a graph with 9 vertices which contains at least one negative cycle.
- 2 Demonstrate the application of breadth-first search on your graph and illustrate the detection of the negative cycle(s).
- 3 Describe a general algorithm to compute all negative cycles in a directed weighted graph.
- 4 Express the cost of the algorithm as $O(f(n, m))$ where n is the number of vertices and m is the number of edges in the graph.

Read section 6.10 in the textbook.

the minimum cost or shortest path problem

If the graph $G = (V, E)$ has no negative cycles,
given $s, t \in V$, find a path P from the origin s to destination t
such that

$$\sum_{(i,j) \in P} c_{i,j}$$

is as small as possible for any path from s to t .

If there is a negative cycle, then the minimum cost is $-\infty$.

In what follows, we assume the input graph has no negative cycles.

Subset Sum; Shortest Paths

1 Subset Sum and Knapsack

- a scheduling problem
- only natural numbers in the input data
- the subset sum algorithm
- the knapsack problem

2 Shortest Paths in a Graph

- directed graphs with negative weights
- the Bellman-Ford algorithm

an important observation

Lemma (8)

Consider $G = (V, E)$, $\#V = n$, and $s, t \in V$. If G has no negative cycles, then there is a shortest path from s to t with no repeat nodes.

Proof. Let P denote the shortest path from s to t .

If P would have repeat nodes, then there are two cases to consider.

- 1 Either, we could remove the repeat nodes and obtain a path that is shorter, which contradicts that P is the shortest path.
- 2 Or, if removing repeat nodes results in a path that is longer, then G has negative cycles, which leads also to a contradiction.

Thus P has no repeat nodes.

Q.E.D.

on the length of the shortest path

Lemma (8)

Consider $G = (V, E)$, $\#V = n$, and $s, t \in V$. If G has no negative cycles, then there is a shortest path from s to t with no repeat nodes.

No repeat nodes in a shortest path of a graph of n vertices implies:

Corollary (9)

Consider $G = (V, E)$, $\#V = n$, and $s, t \in V$. If G has no negative cycles, then there is a shortest path from s to t that has at most $n - 1$ edges.

The important observation bounds the length of the shortest path.

defining the subproblems

The important observation bounds the length of the shortest path.

Let t be the destination vertex in the graph G .

Denote $\text{OPT}(i, v)$ the minimum cost of

- 1 a path from v to t ,
- 2 of at most i edges.

We want to compute $\text{OPT}(n - 1, s)$.

We need to express $\text{OPT}(i, v)$ using smaller subproblems.

deriving a recurrence

Let P be the optimal path with minimum cost $\text{OPT}(i, v)$.

We have one of the following:

- ① either P uses at most $i - 1$ edges, and then

$$\text{OPT}(i, v) = \text{OPT}(i - 1, v),$$

- ② or P uses i edges and the first edge is (v, w) , then

$$\text{OPT}(i, v) = c_{v,w} + \text{OPT}(i - 1, w).$$

To determine w , we consider all vertices.

Lemma (10)

If $i > 0$, then

$$\text{OPT}(i, v) = \min \left(\text{OPT}(i - 1, v), \min_{w \in V \setminus \{v\}} c_{v,w} + \text{OPT}(i - 1, w) \right).$$

the Shortest-Path algorithm

Shortest-Path($G, s, t, c_{i,j}$)

Input: $G = (V, E)$, a graph with no negative cycles,
 s , the start vertex,
 t , the destination vertex,
 $c_{i,j}$, costs of all edges $(i, j) \in E$.

Output: returns $\text{OPT}(n - 1, s)$.

$n := \#V$

let $M[0 \dots n - 1, 1 \dots n]$ be an array of arrays

$M[0, t] := 0$

for all $v \in V \setminus \{t\}$ do $M[0, v] := \infty$

for $i = 1, 2, \dots, n - 1$ do

 for all $v \in V$ do

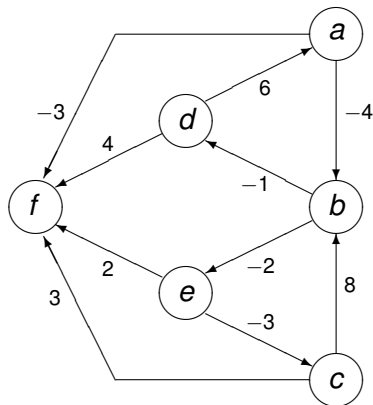
$M[i, v] := \min \left(M[i - 1, v], \min_{w \in V \setminus \{v\}} c_{v,w} + M[i - 1, w] \right)$

return $M[n - 1, s]$

a numerical example

$V = \{a, b, c, d, e, f\}$, $c_{(a,b)} = -4$, $c_{(a,f)} = -3$, $c_{(b,d)} = -1$, $c_{(b,e)} = -2$,
 $c_{(c,b)} = 8$, $c_{(c,f)} = 3$, $c_{(d,a)} = 6$, $c_{(d,f)} = 4$, $c_{(e,c)} = -3$, $c_{(e,f)} = 2$.

Compute all shortest paths to f . Step 0:

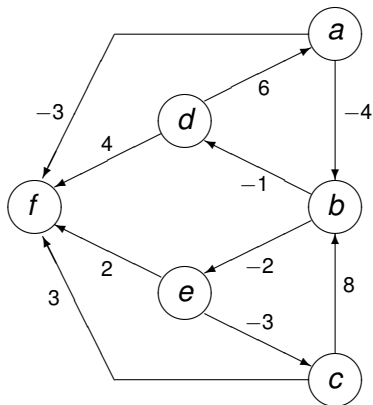


	vertices					
length	a	b	c	d	e	f
5						
4						
3						
2						
1						
0	∞	∞	∞	∞	∞	0

a numerical example

$V = \{a, b, c, d, e, f\}$, $c_{(a,b)} = -4$, $c_{(a,f)} = -3$, $c_{(b,d)} = -1$, $c_{(b,e)} = -2$,
 $c_{(c,b)} = 8$, $c_{(c,f)} = 3$, $c_{(d,a)} = 6$, $c_{(d,f)} = 4$, $c_{(e,c)} = -3$, $c_{(e,f)} = 2$.

Compute all shortest paths to f . Step 1:

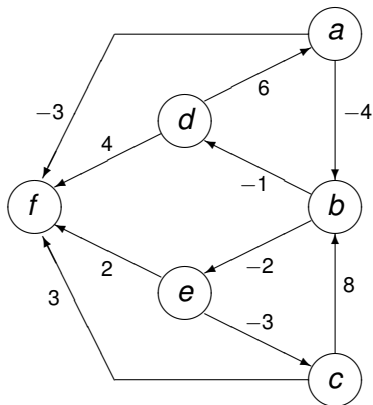


	vertices					
length	a	b	c	d	e	f
5						
4						
3						
2						
1	-3	∞	3	4	2	0
0	∞	∞	∞	∞	∞	0

a numerical example

$V = \{a, b, c, d, e, f\}$, $c_{(a,b)} = -4$, $c_{(a,f)} = -3$, $c_{(b,d)} = -1$, $c_{(b,e)} = -2$,
 $c_{(c,b)} = 8$, $c_{(c,f)} = 3$, $c_{(d,a)} = 6$, $c_{(d,f)} = 4$, $c_{(e,c)} = -3$, $c_{(e,f)} = 2$.

Compute all shortest paths to f . Step 2:

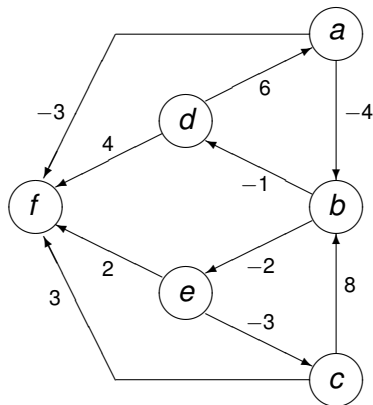


	vertices					
length	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
5						
4						
3						
2	-3	0	3	3	0	0
1	-3	∞	3	4	2	0
0	∞	∞	∞	∞	∞	0

a numerical example

$V = \{a, b, c, d, e, f\}$, $c_{(a,b)} = -4$, $c_{(a,f)} = -3$, $c_{(b,d)} = -1$, $c_{(b,e)} = -2$,
 $c_{(c,b)} = 8$, $c_{(c,f)} = 3$, $c_{(d,a)} = 6$, $c_{(d,f)} = 4$, $c_{(e,c)} = -3$, $c_{(e,f)} = 2$.

Compute all shortest paths to f . Step 3:

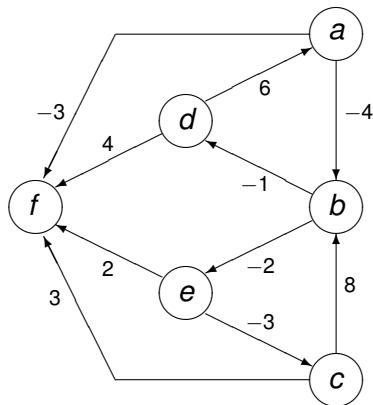


	vertices					
length	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
5						
4						
3	-4	-2	3	3	0	0
2	-3	0	3	3	0	0
1	-3	∞	3	4	2	0
0	∞	∞	∞	∞	∞	0

a numerical example

$V = \{a, b, c, d, e, f\}$, $c_{(a,b)} = -4$, $c_{(a,f)} = -3$, $c_{(b,d)} = -1$, $c_{(b,e)} = -2$,
 $c_{(c,b)} = 8$, $c_{(c,f)} = 3$, $c_{(d,a)} = 6$, $c_{(d,f)} = 4$, $c_{(e,c)} = -3$, $c_{(e,f)} = 2$.

Compute all shortest paths to f . Step 4:

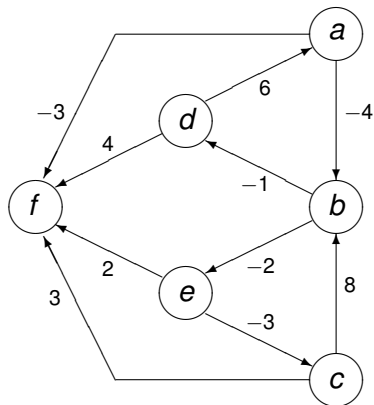


	vertices					
length	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
5						
4	-6	-2	3	2	0	0
3	-4	-2	3	3	0	0
2	-3	0	3	3	0	0
1	-3	∞	3	4	2	0
0	∞	∞	∞	∞	∞	0

a numerical example

$V = \{a, b, c, d, e, f\}$, $c_{(a,b)} = -4$, $c_{(a,f)} = -3$, $c_{(b,d)} = -1$, $c_{(b,e)} = -2$,
 $c_{(c,b)} = 8$, $c_{(c,f)} = 3$, $c_{(d,a)} = 6$, $c_{(d,f)} = 4$, $c_{(e,c)} = -3$, $c_{(e,f)} = 2$.

Compute all shortest paths to f . Step 5:



	vertices					
length	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
5	-6	-2	3	0	0	0
4	-6	-2	3	2	0	0
3	-4	-2	3	3	0	0
2	-3	0	3	3	0	0
1	-3	∞	3	4	2	0
0	∞	∞	∞	∞	∞	0

recovering the shortest paths

Exercise 5: on the example with

$V = \{a, b, c, d, e, f\}$, $c_{(a,b)} = -4$, $c_{(a,f)} = -3$, $c_{(b,d)} = -1$, $c_{(b,e)} = -2$,
 $c_{(c,b)} = 8$, $c_{(c,f)} = 3$, $c_{(d,a)} = 6$, $c_{(d,f)} = 4$, $c_{(e,c)} = -3$, $c_{(e,f)} = 2$,
demonstrate the recovery of all shortest paths to f
given the output table M .

Formulate the algorithm to recover the optimal solution, given M .

What is the running time of this algorithm?

analysis of the algorithm

Theorem (11)

The Shortest-Path algorithm correctly computes the minimum cost of the shortest path between any two vertices in a graph that has no negative cycles in $O(n^3)$ time.

The correctness proof goes by induction, using the recurrence.

For the running time, consider

- 1 the table M has n^2 entries and
- 2 each entry takes $O(n)$ time to compute as there are at most n vertices w to consider.

overview of dynamic programming algorithms

We close chapter 6 on Dynamic Programming.

The algorithms we covered are listed below.

- 1 weighted interval scheduling
- 2 segmented least squares
- 3 sequence alignment
- 4 subset sum
- 5 knapsack
- 6 shortest paths in graphs with negative costs

The dynamic programming algorithms solve a problem by dividing the problem in a polynomial number of subproblems, combining the solutions to the subproblems by a recurrence.